
Algoritmos e Técnicas de Programação

Tema: Introdução a linguagem Python

Professor Wendell Bento Geraldles

Instituto Federal de Goiás
campus Luziânia
<wendell.geraldes@ifg.edu.br>



- 1 Estruturas de decisão no Python
- 2 Estruturas de repetição no Python
- 3 Funções em Python



Estruturas de decisão no Python



Condições lógicas

Python suporta as condições lógicas usuais da matemática:

- É igual a: $a == b$
- Não é igual a: $a != b$
- Menor que: $a < b$
- Menor ou igual a: $a \leq b$
- Maior que: $a > b$
- Maior ou igual a: $a \geq b$

Essas condições podem ser usadas de várias maneiras, mais comumente em "instruções if" e loops. Uma "instrução if" ela é escrita usando a palavra-chave **if** (W3SCHOOLS, 2024).



Estruturas de decisão no Python

Exemplo

```
1 a = 33
2 b = 200
3 if b > a:
4     print("b é maior que a")
```

Neste exemplo, usamos duas variáveis, *a* e *b*, que são usadas como parte da instrução *if* para testar se *b* é maior que *a*. Como *a* é 33 e *b* é 200, sabemos que 200 é maior que 33 e, portanto, imprimimos na tela que "b é maior que a".



Estruturas de decisão no Python

Recuo

Python depende de recuo (espaço em branco no começo de uma linha) para definir o escopo do código. Outras linguagens de programação geralmente usam chaves para esse propósito.

```
5 a = 33
6 b = 200
7 if b > a:
8     print("b é maior que a") #aqui vai dar erro
```



Estruturas de decisão no Python

Elif

A palavra-chave **elif** é a maneira do Python dizer "se as condições anteriores não forem verdadeiras, então tente esta condição".

```
9 a = 33
10 b = 33
11 if b > a:
12     print("b é maior que a")
13 elif a == b:
14     print("a e b são iguais")
```

Neste exemplo, a é igual a b , então a primeira condição não é verdadeira, mas a condição elif é verdadeira, então imprimimos na tela que "a e b são iguais".



else

Você também pode ter um **else** sem o **elif**:

```
15 a = 33
16 b = 33
17 if b > a:
18     print("b é maior que a")
19 else a == b:
20     print("a e b são iguais")
```



Estruturas de decisão no Python

Modo curto

Se você tiver apenas uma instrução para executar, você pode colocá-la na mesma linha da instrução `if`.

```
21 if a > b: print("a is greater than b")
```

Se..Senão

Se você tiver apenas uma instrução para executar, uma para `if` e uma para `else`, você pode colocar tudo na mesma linha.

```
22 a = 2  
23 b = 330  
24 print("A") if a > b else print("B")
```

Essa técnica é conhecida como Operadores Ternários ou Expressões Condicionais .



Estruturas de decisão no Python

Se..Senão

Você também pode ter várias instruções **else** na mesma linha:

```
25 a = 330
26 b = 330
27 print("A") if a > b else print("=") if a == b else print("B")
```

and

A palavra-chave **and** é um operador lógico e é usada para combinar instruções condicionais:

```
28 a = 200
29 b = 33
30 c = 500
31 if a > b and c > a:
32     print("Ambas as condições são verdadeiras")
```



Estruturas de decisão no Python

or

A palavra-chave **or** é um operador lógico e é usada para combinar instruções condicionais:

```
33 a = 200
34 b = 33
35 c = 500
36 if a > b or a > c:
37     print("Pelo menos uma das condições é verdadeira")
```



Estruturas de decisão no Python

not

A palavra-chave **not** é um operador lógico e é usada para reverter o resultado da instrução condicional:

```
38 a = 33
39 b = 200
40 if not a > b:
41     print("a NÃO é maior que b")
```



SE aninhado

Você pode ter instruções **if** dentro de instruções **if**, o que é chamado de instruções aninhadas **if**.

```
42 x = 41
43
44 if x > 10:
45     print("Acima de dez,")
46     if x > 20:
47         print("e também acima de 20!")
48     else:
49         print("mas não acima de 20.")
```



A declaração pass

As instruções if não podem estar vazias, mas se por algum motivo você tiver uma instrução if sem conteúdo, insira a palavra **pass** para evitar erros.

```
50 a = 33
51 b = 200
52
53 if b > a:
54     pass
```



Estruturas de repetição no Python



Estruturas de repetição no Python

Laços no Python

A linguagem Python possui dois tipos de laços de repetição:

- While (Enquanto)
- For (Para)

```
55 #Imprima i enquanto i for menor que 6
56
57 i = 1
58 while i < 6:
59     print(i)
60     i += 1
```

Importante

Você precisa incrementar o i senão o laço será infinito.



A declaração break

O comando **break** faz o laço parar mesmo se a condição while for verdadeira.

```
61 #Saia do loop quando i for 3
62
63 i = 1
64 while i < 6:
65     print(i)
66     if i == 3:
67         break
68     i += 1
```



A declaração continue

O comando **continue** faz o laço parar na iteração atual e continuar com a próxima.

```
69 #Continue para a próxima iteração se i for 3
70
71 i = 0
72 while i < 6:
73     i += 1
74     if i == 3:
75         continue
76     print(i)
```



A declaração else

O comando **else** faz um bloco de comandos ser executado uma vez quando o teste lógico não for mais verdadeiro.

```
77 #Imprima uma mensagem quando a condição for falsa
78
79 i = 1
80 while i < 6:
81     print(i)
82     i += 1
83 else:
84     print("i não é maior que 6")
```



Estruturas de repetição no Python

A estrutura de repetição for

Um loop **for** é usado para iterar sobre uma sequência (pode ser uma lista, uma tupla, um dicionário, um conjunto ou uma string)

```
85 #Imprima cada fruta em uma lista de frutas
86
87 frutas = ["maçã", "banana", "cereja"]
88 for x in frutas:
89     print(x)
```



Estruturas de repetição no Python

Looping através de uma string

Mesmo as strings sendo objetos iteráveis, elas contêm uma sequência de caracteres:

```
90 #Faça um loop pelas letras da palavra "banana"
91
92 for x in "banana":
93     print(x)
```



A declaração break

Com a instrução **break** podemos parar o loop antes que ele tenha percorrido todos os itens:

```
94 #Sair do loop quando x for "banana"
95
96 frutas = ["maça", "banana", "cereja"]
97 for x in frutas:
98     print(x)
99     if x == "banana":
100         break
```



A declaração continue

Com a instrução **continue** podemos parar a iteração atual do loop e continuar com a próxima:

```
101 #Não imprima banana:
102
103 frutas = ["maçã", "banana", "cereja"]
104 for x in frutas:
105     if x == "banana":
106         continue
107     print(x)
```



A declaração continue

Com a instrução **continue** podemos parar a iteração atual do loop e continuar com a próxima:

```
108 #Não imprima banana:
109
110 frutas = ["maçã", "banana", "cereja"]
111 for x in frutas:
112     if x == "banana":
113         continue
114     print(x)
```



Funções em Python



Funções em Python

O que é uma função?

Uma função é um bloco de código que só é executado quando é chamado. Você pode passar dados, conhecidos como parâmetros, para uma função. Uma função pode retornar dados como resultado.

Como definir funções em Python

Em Python, uma função é definida usando a palavra-chave **def**

```
115 def olaMundo():  
116     print("Olá Mundo!")
```



Funções em Python

Como chamar uma função

Para chamar uma função, use o nome da função seguido por parênteses:

```
117 olaMundo()
```

Argumentos

Informações podem ser passadas para funções como argumentos.

```
118 def bomdia(nome):  
119     print("Bom dia, " + nome)  
120  
121 bomdia("João")  
122 bomdia("Maria")  
123 bomdia("José")
```



Funções em Python

Parâmetro ou Argumento?

Os termos parâmetro e argumento podem ser usados para a mesma coisa: informações que são passadas para uma função.

Número de argumentos

Por padrão, uma função deve ser chamada com o número correto de argumentos. O que significa que se sua função espera 2 argumentos, você tem que chamar a função com 2 argumentos, nem mais, nem menos.

```
124 def nome_sobrenome(nome, sobrenome):  
125     print(nome + " " + sobrenome)  
126  
127 nome_sobrenome("José", "Maria")
```



Valor do parâmetro padrão

O exemplo a seguir mostra como usar um valor de parâmetro padrão. Se chamarmos a função sem argumento, ela usará o valor padrão:

```
128 def meuPais(pais = "Brasil"):
129     print("Eu nasci no(a) " + pais)
130
131 meuPais("França")
132 meuPais("Chile")
133 meuPais()
```



Passando uma lista como argumento

Você pode enviar qualquer tipo de dado de argumento para uma função (string, número, lista, dicionário etc.), e ele será tratado como o mesmo tipo de dado dentro da função. Por exemplo, se você enviar uma Lista como argumento, ela ainda será uma Lista quando chegar à função:

```
134 def frutascitricas(frutas):  
135     for x in frutas:  
136         print(x)  
137  
138 frutas = ["Laranja", "Limão", "Tangerina"]
```



Valores de retorno

Para permitir que uma função retorne um valor, use a instrução **return**:

```
139 def tabuada5(x):  
140     return 5 * x  
141  
142 print(tabuada5(3))  
143 print(tabuada5(5))  
144 print(tabuada5(9))
```



 W3SCHOOLS. **Python**. W3C, 2024. Disponível em: <<https://www.w3schools.com>>.

